Problem Statement 12: DemoBlaze – Selenium-Java Test Automation Framework for an Electronics Demo Store

---

1. Project Overview

The DemoBlaze Automation Framework is an end-to-end test automation solution developed for the DemoBlaze e-commerce platform. It is built using Selenium WebDriver with Java and uses TestNG as the test framework.

The framework is designed to automate and validate major e-commerce workflows such as:

- User authentication
- Product browsing and category filtering
- Shopping cart operations
- Order placement
- Form validation scenarios

It follows a structured and modular approach using the Page Object Model (POM) to ensure scalability, maintainability, and reusability.

---

2. Objectives

- Automate critical user flows of the DemoBlaze application
- Ensure modular and maintainable framework design
- Implement data-driven testing where required
- Integrate reporting using Extent Reports
- Handle dynamic elements and alerts effectively
- Ensure test isolation using unique test data
- Enable parallel execution for faster testing

---

3. Technology Stack

Component	Technology
Programming Language	**Java**
Automation Tool	**Selenium WebDriver (4.34.0)**
Test Framework	**TestNG (7.10.2)**
Build Tool	**Maven**
Driver Management	**WebDriverManager (5.9.2)**
Data Handling	**Apache POI (5.4.1), JSON**
Reporting	**Extent Reports**

---

4. Framework Architecture

The framework follows a layered architecture for better separation of concerns:

Layers

1. **Test Layer**
    - Contains all test classes (AuthTest, CartTest, etc.)
    - Implements test scenarios

2. **Page Layer**
    - Contains page object classes
    - Handles UI interactions and element locators
3. **Base Layer**
    - Common setup and teardown logic
    - WebDriver initialization
4. **Utility Layer**
    - Reusable helper classes:
        - ConfigReader
        - ScreenshotUtils
        - JsonUtil
        - WaitUtils
        - ExtentManager
5. **Data Layer**
    - External data sources and configurations

This layered approach ensures:

- High maintainability
- Reusability
- Scalability

---

5. Project Structure

```
demoblaze-automation/
├── pom.xml
├── testng.xml
├── README.md
├── src/
│   ├── main/
│   │   └── java/
│   │       └── com/srm/hackathon/demoblaze/
│   │           ├── base/
```

```
│   │           │   └── BasePage.java
│   │           ├── factory/
│   │           │   ├── DriverFactory.java
│   │           │   └── DriverManager.java
│   │           ├── pages/
│   │           │   ├── CartPage.java
│   │           │   ├── HomePage.java
│   │           │   ├── LoginModalPage.java
│   │           │   ├── OrderPage.java
│   │           │   ├── ProductDetailPage.java
│   │           │   └── ProductPage.java
│   │           └── utils/
│   │               ├── ConfigReader.java
│   │               ├── ExtentManager.java
│   │               ├── JsonUtil.java
│   │               ├── ScreenshotUtils.java
│   │               └── WaitUtils.java
│   │
│   └── test/
│       ├── java/
│       │   └── com/srm/hackathon/demoblaze/
│       │       ├── base/
│       │       │   └── BaseTest.java
```

```
|       |       ├── dataprovider/
|       |       |   └── DataProviders.java
|       |       ├── listeners/
|       |       |   └── TestListener.java
|       |       └── tests/
|       |           ├── AuthTest.java
|       |           ├── CartTest.java
|       |           ├── FormValidationTest.java
|       |           ├── OrderTest.java
|       |           └── ProductTest.java
|       └── resources/
|
├── target/
|   ├── classes/
|   └── test-classes/
|
├── reports/
|   └── ExtentReport.html
|
├── screenshots/
|   └── [Test execution screenshots]
└── test-output/
    └── [TestNG reports]
```

---

6. Design Approach

6.1 Page Object Model (POM)

- Each page is represented as a separate class
- Locators are centralized
- Methods represent user actions

Benefits:

- Code reusability
- Easy maintenance
- Reduced duplication

6.2 Base Classes

- **BasePage**
  - Provides reusable methods (click, sendKeys, waits)
- **BaseTest**
  - Handles WebDriver setup and teardown
  - Manages browser lifecycle

6.3 Utility Classes

- DriverFactory → Creates WebDriver instances
- DriverManager → Manages driver lifecycle
- ConfigReader → Reads configuration properties
- WaitUtils → Explicit wait handling
- ScreenshotUtils → Captures failure screenshots
- ExtentManager → Generates reports
- JsonUtil → Handles JSON test data

6.4 Test Listeners

- **TestListener**
  - Captures test execution logs

    - Takes screenshots on failures
    - Integrates Extent Reports

---

7. Test Modules

7.1 Authentication Module (AuthTest)

Test Cases:

- verifyUserRegistration
- verifySuccessfulLogin

Approach:

- Uses `LoginModalPage` for interactions
- Handles alert popups
- Validates login success via navbar

7.2 Product Module (ProductTest)

Test Cases:

- verifyPhonesCategory
- verifyLaptopsCategory
- verifyMonitorsCategory

Approach:

- Performs login before testing
- Uses category filters
- Validates product listings

7.3 Cart Module (CartTest)

Test Cases:

- verifyAddSingleProduct

- verifyAddMultipleProducts
- verifyDeleteProductFromCart
- verifyUpdateProductQuantity
- verifyEmptyCartMessage

Approach:

- Uses dynamic username generation
- Registers new user for each test
- Validates cart updates and totals

7.4 Order Module (OrderTest)

Test Cases:

- verifyPlaceOrderSuccess
- verifyOrderDetails
- verifyOrderID
- verifyOrderWithMultipleProducts

Approach:

- Adds products before checkout
- Fills order form
- Validates confirmation message

7.5 Form Validation Module (FormValidationTest)

Test Cases:

- verifySignupWithExistingUser
- verifyLoginWithEmptyUsername
- verifyLoginWithEmptyPassword
- verifyLoginWithInvalidCredentials

Approach:

- Validates error messages
- Tests edge cases
- Ensures proper input handling

---

8. Data-Driven Testing

- Uses dynamic test data (timestamps for usernames)
- DataProviders enable parameterized testing
- Configuration handled via properties files

Benefits:

- Avoids test conflicts
- Improves scalability

---

9. Reporting

Extent Reports is used for detailed reporting.

Features:

- Pass/Fail status
- Execution time
- Screenshots on failure
- Logs and system info

Location:

reports/ExtentReport.html

---

10. Error Handling

- Explicit waits for synchronization
- Alert handling for popups
- Exception handling using try-catch
- Logging for debugging
- Screenshot capture on failure

---

11. Parallel Execution

- Configured using TestNG

- Thread count: 3
- Improves execution speed
- Ensures test independence

---

12. Assumptions

- Application is stable and accessible
- Alerts are JavaScript-based
- Unique user is created for each test
- Browser has internet access

---

13. Limitations

- No API testing included
- Limited cross-browser testing
- No mobile automation
- Some validations depend on UI behavior

---

14. Future Enhancements

- Cross-browser execution (Chrome, Firefox, Edge)
- CI/CD integration (Jenkins, GitHub Actions)
- Docker-based execution
- API automation integration
- Retry mechanism for flaky tests
- Advanced reporting dashboards

---

15. Conclusion

The DemoBlaze Automation Framework provides a scalable, maintainable, and efficient solution for testing e-commerce applications. By leveraging industry best practices such as Page Object Model, modular architecture, data-driven testing, and detailed reporting, the framework ensures high-quality test automation.

AUTHOR : Bhavya Sree Kasa